Installation et utilisation — CVRPTW ALNS

Version documentée

Ce guide décrit le fichier `CVRPTW_ALNS.jl` transmis par l'utilisateur : 1 199 lignes. Le code n'a pas été modifié ni exécuté dans Julia pendant la rédaction. L'analyse est statique ; aucun temps de calcul ni besoin maximal de RAM n'a été mesuré.

1. Fonction du logiciel

Résoudre heuristiquement un problème de tournées de véhicules avec capacité et fenêtres de temps (CVRPTW), à partir d'une instance Solomon/Homberger compatible avec le lecteur.

L'ALNS détruit une partie d'une solution, réinsère les clients puis accepte ou rejette la proposition. Plusieurs chaînes CPU coopèrent en publiant leurs meilleures solutions. Le classement interne minimise d'abord le nombre de véhicules, puis la distance. L'arrêt facultatif sur gap porte uniquement sur la distance.

Ce programme ne certifie ni l'optimalité ni l'atteinte d'un gap donné.

2. Installation de Julia sous Windows

Installation recommandée : Juliaup

La documentation officielle recommande Juliaup, gestionnaire qui installe Julia et permet de gérer ses versions. Dans PowerShell, exécuter :

```
winget install --name Julia --id 9NJNWW8PVKMN -e -s msstore
```

Accepter les demandes de confirmation affichées par l'installateur. L'installation peut aussi être lancée depuis le Microsoft Store en recherchant Julia. Si le Store est bloqué ou indisponible, consulter la page officielle d'installation pour les autres méthodes ; ne pas télécharger un exécutable depuis une source inconnue.

Fermer puis rouvrir PowerShell et, si nécessaire, VS Code après l'installation.

Vérifier l'installation

```
julia --version
juliaup status
```

Pour vérifier les threads disponibles dans une session lancée avec quatre threads :

```
julia --threads 4 -e "println(Threads.nthreads()); versioninfo()"
```

La première ligne doit indiquer `4`. Le détail de version permet de conserver une trace de l'environnement utilisé pour les essais.

Gestion des versions

```
juliaup update
```

Cette commande met à jour les versions gérées. Pour conserver explicitement la version 1.11.7 aperçue dans les premières traces de ce projet :

```
juliaup add 1.11.7
julia +1.11.7 --version
```

Il n'est pas nécessaire d'imposer cette ancienne version si la version installée fonctionne. La compatibilité du programme n'a pas été testée ici sur toutes les versions de Julia.

Dépendances

Le script importe seulement `Random`, `Printf` et `Base.Threads`, fournis avec Julia. Aucun `Pkg.add`, Python, CUDA, solveur commercial ou compilation C/C++ n'est nécessaire pour ce fichier.

`--project=.` sélectionne l'environnement du dossier courant. Aucun fichier Project.toml spécifique n'est requis par les imports de ce script ; cette option peut être conservée dans les commandes habituelles.

3. Spécifications techniques

Environnement et matériel

Élément	Exigence ou recommandation
Langage	Julia ; version 1.11.7 présente dans les traces antérieures de l'utilisateur.
Système visé par ce guide	Windows avec PowerShell. Le code ne comporte pas d'appel système propre à Windows ; les autres systèmes n'ont pas été testés ici.
Architecture conseillée	Installation 64 bits adaptée à la machine.
CPU	Exécution sur CPU, une chaîne possible avec un thread ; plusieurs threads pour une exécution simultanée de plusieurs chaînes.
Configuration de départ proposée	4 threads Julia et 4 chaînes ; à comparer expérimentalement à 2 threads/2 chaînes.
RAM	Aucun minimum mesuré. 8 Go de RAM système ou davantage constituent une recommandation pratique, pas une exigence validée du programme. La RAM installée de l'utilisateur n'est pas connue.
GPU	Non utilisé par ce script. Aucune carte graphique dédiée requise.

Élément	Exigence ou recommandation
Stockage	Place pour Julia, les instances et les résultats ; aucun volume total minimal mesuré.
Connexion réseau	Utile à l'installation/mise à jour de Julia ; le calcul ne fait pas d'appel réseau.
Droits sur les fichiers	Lecture de l'instance/référence et écriture dans le dossier de l'instance.

Processeur communiqué par l'utilisateur

Intel Core i3-1315U : 6 cœurs physiques (2 cœurs performance et 4 cœurs efficaces), 8 threads, fréquence turbo maximale annoncée de 4,50 GHz. Le `1,20 GHz` affiché dans le nom du processeur ne signifie pas que tous les calculs s'effectuent constamment à cette fréquence. La fréquence soutenue dépend notamment de l'alimentation et des limites thermiques.

Le modèle de CPU ne suffit pas à identifier tous les composants de l'ordinateur : RAM, stockage et éventuel GPU dédié restent à vérifier séparément.

Quatre chaînes ne garantissent pas une accélération par quatre. Les chaînes partagent le processeur, la mémoire et certains accès verrouillés.

Données et modèle mathématique

- Entrée : format texte avec en-tête `NUMBER` et `CAPACITY` sur une même ligne.
- Une ligne client contient sept nombres : identifiant, x, y, demande, début de fenêtre, fin de fenêtre, durée de service.
- Identifiants consécutifs : dépôt 0, puis clients 1 à n.
- Distances euclidiennes non arrondies calculées en Float64.
- Temps de déplacement numérique égal à la distance ; pas de vitesse supplémentaire ni de trafic dépendant de l'heure.
- Une route part du dépôt et y revient ; une visite par client et respect de la capacité.
- Temps de service pris en compte chez les clients. La durée de service du dépôt n'est pas ajoutée par le calcul de route.
- Tolérance numérique `EPS = 1e-7`.
- Matrice complète de distances : environ `8 × (n+1)^2` octets de données brutes, sans les structures annexes. Pour 600 clients, environ 2,89 Mo décimaux, soit 2,76 Mio. Les solutions, copies privées, allocations temporaires et le runtime Julia demandent de la mémoire supplémentaire.
- Jusqu'à 30 voisins clients précalculés par nœud.

4. Organisation des fichiers

```
RO/
  CVRPTW_ALNS.jl
  C1_2_1.txt
  C1_2_1.sol                  (facultatif : référence)
```

Ne pas donner le `.sol` de référence comme premier argument. C'est le contenu du fichier qui détermine son rôle, pas uniquement son extension.

5. Ouvrir le dossier et lancer

```
cd "C:\Users\lucas\OneDrive\Documents\Perso\RO"
```

Toutes les itérations : aucun gap cible par défaut

```
julia --threads 4 --project=. -O3 .\CVRPTW_ALNS.jl .\C1_2_1.txt 50000 4
```

Chaque chaîne effectue 50 000 itérations. Avec quatre chaînes, le budget total est donc 200 000 itérations. La coopération ne transforme pas ce budget en 50 000 itérations réparties.

Arrêt facultatif à 6 % de gap distance

```
julia --threads 4 --project=. -O3 .\CVRPTW_ALNS.jl .\RC1_6_2.txt 50000 4 0.06
```

À 3 %, remplacer `0.06` par `0.03`. Un seuil de 10 % s'écrit `0.10`, pas `10`.

Fixer la graine sans seuil d'arrêt

```
julia --threads 4 --project=. -O3 .\CVRPTW_ALNS.jl .\C1_2_1.txt 50000 4 Inf 123
```

`Inf` désactive le seuil et permet de conserver la position de l'argument seed.

Syntaxe des arguments

```
julia [options Julia] script.jl instance iterations [chaines] [gap] [seed]
```

Argument	Valeur par défaut	Rôle
instance	obligatoire	Fichier de données.
iterations	obligatoire en ligne de commande	Maximum par chaîne.
chaines	4	Nombre de chaînes de recherche.
gap	Inf	Aucun arrêt sur gap si absent ou égal à Inf.
seed	42	Graine de départ ; chaque chaîne utilise seed + 7919 × numéro.

Les options `--threads`, `--project` et `-O3` doivent précéder le nom du script. Le nombre de chaînes après le script ne crée pas automatiquement les threads Julia.

6. Références et calcul du gap

Pour une instance `C1_2_1.txt`, le script cherche `C1_2_1.sol`, `C1_2_1_ref.sol` ou `C1_2_1_reference.sol`, dans le dossier de l'instance ou ses sous-dossiers `reference` et `bks`. La comparaison des noms ne distingue pas la casse. Si plusieurs références conviennent, la première référence exploitable rencontrée est retenue : éviter les doublons contradictoires.

Lignes de coût reconnues notamment :

```
Cost: 12345.67
Distance = 12345.67
Objective: 12345.67
```

Le coût doit être positif et fini. Les décimales avec virgule sont acceptées, mais pas les séparateurs de milliers. La ligne doit correspondre entièrement au format attendu.

Sans coût reconnu, le lecteur tente les lignes `Route #1: 12 45 8`, valide les routes et recalcule la distance. Un dépôt 0 peut figurer aux extrémités.

Une ligne de coût directement reconnue est utilisée sans validation de l'ensemble des routes de référence : vérifier qu'il s'agit bien d'une distance calculée avec la même convention que le solveur. Une référence personnelle n'est pas nécessairement un meilleur résultat connu publié.

```
gap (%) = 100 × (distance_obtenue / distance_reference − 1)
```

Le gap d'arrêt ne tient pas compte du nombre de véhicules. Le classement des meilleures solutions utilise au contraire le couple `(véhicules, distance)`. Ces deux critères peuvent sélectionner des solutions différentes ; en cas d'arrêt sur gap, le script conserve la solution ayant déclenché cet arrêt.

Référence absente ou illisible

Un avertissement apparaît. L'arrêt sur gap est désactivé, toutes les itérations sont effectuées, puis la meilleure solution est enregistrée. Sans seuil demandé, une référence lisible sert seulement au suivi et au bilan du gap.

7. Fonctionnement parallèle et paramètres internes

- Tâches lancées avec `Threads.@spawn` et attendues par `@sync`.
- Instance partagée en lecture ; solutions et générateurs aléatoires privés.
- Meilleur global protégé par `ReentrantLock` et drapeau d'arrêt `Atomic{Bool}`.
- Consultation/migration toutes les 200 itérations ; adoption conditionnelle dans la solution courante.
- Cinq destructions : aléatoire, retrait selon le gain, route entière, proximité espace/temps, segments.
- Deux réparations : gloutonne diversifiée et regret-2.

- Réglage des poids toutes les 100 itérations, plancher 0,1.
- Relocation toutes les 25 itérations si la réparation a réussi ; au plus 100 clients testés et 15 mouvements acceptés par appel.
- Redémarrage après 1 500 itérations sans amélioration locale.
- Pénalité d'ouverture de route dans les insertions : 1 000 ; ce n'est pas une pénalité du classement final.

L'arrêt global est contrôlé au début de chaque itération ; les tâches déjà engagées dans une itération terminent celle-ci. La construction initiale ne teste pas le drapeau à chaque client.

8. Résultats et sauvegarde

Dans le dossier de l'instance :

- `C1_2_1_alns.sol` : routes et distance à six décimales ;
- `C1_2_1_alns_resume.txt` : durée, flotte, distance, référence éventuelle, gap, statut et compteurs par chaîne.

Le fichier de référence n'est pas remplacé. Les résultats portant déjà ces noms sont remplacés lors d'un nouveau lancement. Éviter deux programmes simultanés sur la même instance : ils utiliseraient les mêmes noms de sortie et de fichier temporaire.

Au seuil atteint, le `.sol` est écrit avant de lever le drapeau d'arrêt. Sans seuil atteint, la sauvegarde se fait au terme du calcul. Une nouvelle validation précède l'écriture finale.

Aucune sauvegarde de secours sur Ctrl+C, panne ou erreur non gérée n'est implémentée. Le résumé est écrit après la fin de toutes les tâches. Le temps affiché inclut notamment la lecture/préparation et la recherche, mais n'inclut pas toute la durée du processus Julia ni l'écriture finale du résumé.

9. Dépannage

`julia` non reconnu

Rouvrir le terminal/VS Code après installation ; vérifier `julia --version`. Si nécessaire, consulter la documentation officielle plutôt que modifier le PATH au hasard.

En-tête NUMBER / CAPACITY introuvable

Le premier argument est-il l'instance `.txt`, et non la référence `.sol` ? Si c'est l'instance, vérifier que son format est accepté par le lecteur.

Identifiants non consécutifs

Le code exige les identifiants 0, 1, …, n. Ce lecteur ne gère pas une numérotation arbitraire.

Référence non reconnue

Vérifier emplacement, nom et ligne de coût. Une référence trouvée mais incompatible désactive l'arrêt sur gap ; le calcul reste possible.

Aucun affichage pendant un moment

Les lignes affichées correspondent surtout aux améliorations globales, pas à chaque itération. L'absence de message ne prouve pas un blocage.

Même graine, résultat différent

La coopération dépend de l'ordonnancement des tâches : la même graine ne suffit pas à garantir une exécution reproductible à l'identique en parallèle.

10. Limites de cette version à connaître

1. Sans gap cible (`Inf`) mais avec référence, une branche de `run_chain` appelle encore `publish!` pour des solutions acceptées qui ne battent pas le meilleur local : la condition distance ≤ référence × (1 + Inf) est toujours vraie pour une distance finie. L'arrêt reste désactivé dans `publish!`, mais ces validations/copies supplémentaires peuvent ralentir l'exécution.
2. Le tri initial appelle `rand(rng)` dans la fonction de clé : les clés de départage aléatoires ne sont pas précalculées et fixes pour chaque client. Une amélioration future serait de tirer ces clés une seule fois avant le tri.
3. Le budget minimal par chaîne est un nombre d'itérations ; aucune limite en secondes n'est implémentée.
4. La qualité et le temps d'atteinte doivent être mesurés : les anciennes promesses de gains linéaires ou de gap garanti ne sont pas des garanties du programme.

11. Sources et provenance

- Fichier source utilisateur : `CVRPTW_ALNS.jl`, 1 199 lignes, analysé sans modification.
- Installation officielle Julia : https://docs.julialang.org/en/v1/manual/installation/
- Téléchargements Julia : https://julialang.org/downloads/
- Gestionnaire Juliaup : https://github.com/JuliaLang/juliaup
- Fiche processeur : Intel Core i3-1315U, référence Intel 232136 (caractéristiques constructeur) ; informations de modèle également décrites sur https://laptopmedia.com/processor/intel-core-i3-1315u/

Les conseils de RAM et de nombre de chaînes sont des recommandations de départ, non des mesures de ce programme.